PRACTICAL TAKE-HOME ASSIGNMENT

nopCommerce QA Automation

Software Quality Assurance Analysis, Test Design and Selenium Implementation

Report detail	Value
Student	Bavindu Shamen
Application	nopCommerce public demo store
Technology	Java 21 | Selenium 4 | TestNG 7 | Maven | Page Object Model
Date	12 August 2026
Repository	nopcommerce-qa-automation

Submission note: the public demo's Cloudflare verification blocked the recorded Selenium run. This is reported as an environment limitation, not a pass or product defect.

Executive summary

This project evaluates the customer-facing nopCommerce demo store and implements a maintainable browser-automation suite for five high-value regression scenarios. The work follows a traceable sequence: application selection, requirement analysis, manual scenario design, automation decisions, framework implementation, execution, and controlled debugging.

Eighteen manual scenarios cover registration, authentication, search, catalog, cart, comparison, and guest checkout. Five were automated using Java, Selenium WebDriver, TestNG, Maven, and Page Object Model. The project includes configuration overrides, explicit waits, isolated browser sessions, and failure screenshots.

The source compiled successfully and the controlled debugging test passed after its intentional assertion mismatch was corrected. A live full-suite attempt was blocked before application interaction by nopCommerce's Cloudflare security-verification page. The limitation is preserved transparently and the suite remains ready for an authorised rerun.

1. Application selection and scope

The selected system is the official public demonstration storefront at https://demo.nopcommerce.com/. It models realistic business-to-consumer journeys without requiring access to a production system or real financial information.

Criterion	Assessment
Functional breadth	Authentication, search, product detail, cart, comparison, and checkout support broad test design.
Automation fit	Distinct pages and stable user-visible outcomes support reusable Page Objects.
Safety	Synthetic customer and address data can be used in a public demo environment.
Positive/negative coverage	Forms and searches provide successful, invalid, and empty-result behaviours.

In scope are the desktop Chrome storefront, customer-visible validation, cart and comparison state, and guest checkout confirmation. Administration, database/API validation, real payment processing, email delivery, load/security testing, and pixel-perfect visual comparison are excluded.

2. Requirement and risk analysis

ID	Area	Expected behaviour
FR-01	Navigation	Reach home, category, search, account, cart and comparison functions.
FR-02	Registration	Validate mandatory fields; accept unique users; reject duplicate email.
FR-03	Authentication	Accept valid credentials, reject invalid credentials, and support logout.
FR-04	Discovery	Search and browse products; explain empty results; support listing controls.

ID	Area	Expected behaviour
FR-05	Product	Show product details and enforce required configuration.
FR-06	Cart	Maintain products, quantities, prices, removal, and calculated totals.
FR-07	Comparison	Add, display, remove, and clear comparable products.
FR-08	Checkout	Validate terms and customer data; complete guest order confirmation.

Risk	Impact	Mitigation
Shared demo data and hourly reset	High	Create prerequisites per test; use guest and synthetic data.
Catalog/UI changes	High	Keep selectors in Page Objects and catalog values in configuration.
Asynchronous state	High	Use condition-based explicit waits; prohibit fixed sleeps.
Network/demo outage or challenge	High	Classify as environment failure and preserve evidence.
Browser/driver mismatch	High	Use Selenium Manager and record the execution versions.

3. Manual test design

Priorities are risk based: P0 blocks a core journey or checkout; P1 materially affects discovery, account use, or cart integrity; P2 affects a supporting feature. Every scenario has explicit preconditions, data, steps, expected outcome, and evidence guidance in the repository's detailed manual-test document.

ID	Scenario	Priority	Trace
MTS-01	Register with valid unique data	P1	FR-02
MTS-02	Validate required registration fields	P1	FR-02
MTS-03	Reject registration using an existing email	P1	FR-02
MTS-04	Log in with valid credentials	P0	FR-03
MTS-05	Reject invalid login credentials	P0	FR-03
MTS-06	Log out an authenticated customer	P1	FR-03
MTS-07	Search for an existing product	P0	FR-04
MTS-08	Search for a nonexistent product	P1	FR-04
MTS-09	Browse a catalog category	P1	FR-01/04
MTS-10	Sort a product listing by price	P1	FR-04
MTS-11	Add a simple product to the cart	P0	FR-05/06
MTS-12	Add a configured product to the cart	P0	FR-05/06
MTS-13	Update product quantity in the cart	P0	FR-06
MTS-14	Remove a product from the cart	P1	FR-06

ID	Scenario	Priority	Trace
MTS-15	Compare two products	P2	FR-07
MTS-16	Clear the product comparison list	P2	FR-07
MTS-17	Validate checkout terms requirement	P0	FR-08
MTS-18	Complete a valid guest checkout	P0	FR-08

4. Automation decision

Auto ID	Manual	Scenario	Why selected
AUT-01	MTS-05	Invalid login	Critical negative boundary; independent data and stable message.
AUT-02	MTS-07	Existing-product search	Fast, frequent discovery path with relevance assertions.
AUT-03	MTS-11	Add simple product	Critical catalog-to-cart state transition.
AUT-04	MTS-15	Compare two products	Reusable repeated actions and cross-page state.
AUT-05	MTS-18	Guest checkout	Revenue-critical end-to-end integration journey.

The other thirteen scenarios remain manual in this five-test scope because of persistent-account dependencies, duplicated mechanics, volatile pricing/configuration, lower business impact, or stronger exploratory value. MTS-13 quantity recalculation is the recommended next automation candidate.

5. Automation framework

Package	Responsibility
config	Loads property defaults with command-line overrides.
driver	Creates and closes WebDriver through Selenium Manager.
base	Creates a clean browser before each test and always tears it down.
pages	Owns private locators, explicit waits, and user actions.
models	Represents immutable synthetic checkout data.
tests	Expresses scenario intent and owns TestNG assertions.
listeners	Captures timestamped screenshots when a test fails.

The design intentionally avoids direct element lookup in test classes, fixed sleeps, implicit waits, shared test order, and automatic retries. Mutable product values are held in config.properties. Each selected test receives a new browser session so cookies, cart items, and comparison state cannot leak between scenarios.

Locator preference is stable id, form name, component-scoped CSS, semantic relationship, then business text. ExpectedConditions wait for visibility, clickability, URL changes, selectable options, and user-visible notifications.

6. Automated scenario implementation

ID	Page flow	Principal assertion
AUT-01	HomePage -> LoginPage	Generic unsuccessful-login message is displayed.
AUT-02	HomePage -> SearchResultsPage	Query is retained and a relevant product is returned.
AUT-03	ProductPage -> CartPage	HTC smartphone exists in cart with quantity one.
AUT-04	ProductPage -> CompareProductsPage	HTC smartphone and Apple MacBook Pro are both displayed.
AUT-05	Product -> Cart -> GuestCheckout -> Confirmation	Success message and non-empty order number are displayed.

Synthetic checkout data uses example.com and a non-real telephone number. The test chooses currently enabled shipping and payment methods, avoiding assumptions about a specific demo option. No real payment details are submitted.

7. Execution and results

Environment item	Recorded value
Operating system	macOS 26.5.1, Apple Silicon
Java / Maven	OpenJDK 21.0.12 / Maven 3.9.11
Selenium / TestNG	4.44.0 / 7.12.0
Chrome / driver	151.0.7922.76 / 151.0.7922.138

Check	Status	Evidence
Compile verification	PASS	Main and test sources compiled successfully.
Controlled debugging rerun	PASS	1 test; 0 failures, errors, or skips.
Five live Selenium scenarios	BLOCKED	Cloudflare verification replaced the app before first expected elements.

The live suite result is not reported as a pass. Five screenshots show the same 'Performing security verification' page, and stack traces time out at each test's first nopCommerce locator. This common first divergence classifies the result as an external environment blocker. No attempt was made to bypass the site's security control.

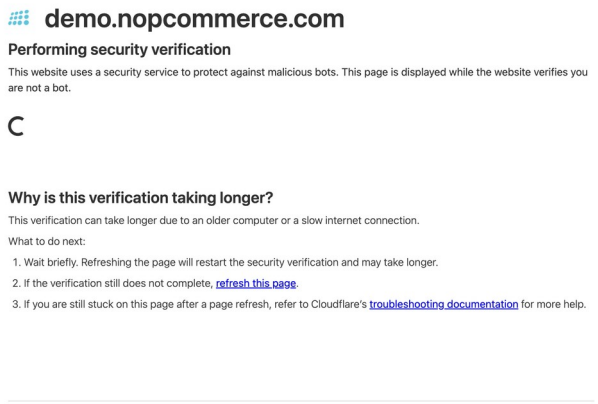

Figure 1. Failure-listener screenshot: nopCommerce security verification replaced the application UI.

8. Intentional failure and debugging

Commit 36f3657 introduced a deterministic assertion failure. A captured generic login error contained 'Login was unsuccessful', while the temporary assertion searched for 'No customer account found'. TestNG reported one test, one failure, and an AssertionError at the assertion line.

Debug step	Evidence and conclusion
Observe	Expected true but found false; failure isolated to the assertion.
Compare	Known input and expected phrase did not describe the same message contract.
Classify	Test defect, not application or runner defect.
Correct	Assert the stable generic phrase: 'Login was unsuccessful'.
Verify	Focused rerun completed with 1 test and 0 failures.

Commit dc535b2 preserves the correction and its explanation. Keeping the failure and fix as consecutive milestones creates reproducible evidence without leaving the current branch broken.

9. Maintainability and limitations

The Page Object boundary localises selector changes, system properties support environment overrides, and isolated sessions protect test independence. The repository's meaningful commit sequence allows the analysis, framework, scenarios, intentional failure, and fix to be reviewed independently.

The main limitation is dependency on a public shared demo whose catalog, reset cycle, availability, and bot protection are outside the project team's control. A dedicated authorised test environment would provide reliable CI execution and safer test-data lifecycle management. Until then, every execution should classify environment failures before interpreting application assertions.

10. Conclusion and next steps

The project fulfils the design and implementation objectives: a justified application choice, requirement analysis, 18 traceable manual scenarios, five risk-based automation decisions, a Java/Selenium/TestNG/POM framework, five automated flows, failure evidence, and a controlled debugging demonstration.

Before final submission, the student should rerun the suite when nopCommerce permits automation, update the execution record with the actual result, capture the required manual evidence, record the five-minute reflection in their own words, confirm lecturer registration of the selected demo if required, and insert the final GitHub and video links.

Appendix A - Repository evidence

Commit	Milestone
566ec2c	Initialize repository governance
1bb6775	Select nopCommerce demo application
2ecc2c1	Analyse requirements and risks
7e28d04	Define 18 manual scenarios
0459016	Select five automation scenarios
cd9ca82	Establish Selenium TestNG POM framework
24d83f0	Automate five regression scenarios
36f3657	Introduce controlled assertion failure
dc535b2	Correct assertion and document debugging
9e05fa2	Add video reflection and viva guides

Appendix B - AI usage disclosure

OpenAI Codex supported assignment decomposition, website comparison, live-flow review, implementation drafts, code review, debugging documentation, and report generation. The student remains responsible for reviewing and understanding all work, validating the live application's expected behaviour, executing the suite, explaining decisions during the viva, and maintaining the full activity log in AI_USAGE.md.